

1 – Boolesche Algebra

Aussagenlogik · Wahrheitstabellen · Rechengesetze · De Morgan · Anwendung in Python und SQL

Boolesche Algebra ist das Rechnen mit Wahrheitswerten. Sie steht am Anfang dieser Woche, weil sie ohne Vorkenntnisse auskommt und weil sie in der täglichen Arbeit eines Datenanalysten permanent vorkommt: jede WHERE-Bedingung in SQL, jede if-Abfrage in Python, jeder Filter in Power BI und jede Maske in Pandas ist ein boolescher Ausdruck. Wer die Gesetze kennt, schreibt kürzere Filter, findet Denkfehler in Bedingungen und versteht, warum eine Abfrage zu viele oder zu wenige Zeilen liefert.

1 – Aussagen und Wahrheitswerte

Eine Aussage im Sinne der Logik ist ein Satz, dem sich eindeutig genau einer von zwei Werten zuordnen lässt: wahr oder falsch. Ein dritter Wert ist nicht vorgesehen. Dieses Prinzip heißt Zweiwertigkeit.

Aussage oder keine Aussage?

Satz	Aussage?	Begründung
Berlin liegt in Deutschland.	ja	eindeutig wahr
$3 + 4 = 8$	ja	eindeutig falsch
Die Bestellung ist bezahlt.	ja	wahr oder falsch, je nach Datensatz
Wie viel kostet das Produkt?	nein	Frage – kein Wahrheitswert
Bitte laden Sie die Datei hoch.	nein	Aufforderung
Dieses Produkt ist teuer.	nein	unscharf – „teuer“ ist nicht definiert

Praxisbezug: Die letzte Zeile ist in der Datenanalyse der Normalfall. Fachliche Anforderungen wie „teure Produkte anzeigen“ sind keine Aussagen. Erst die Übersetzung in $\text{preis} > 100$ macht daraus eine Aussage, die eine Datenbank auswerten kann. Diese Übersetzung ist die eigentliche Arbeit.

Schreibweisen

Aussagen werden mit Kleinbuchstaben abgekürzt, üblich sind a, b, c oder p, q, r. Für die Wahrheitswerte gibt es mehrere gebräuchliche Notationen, die alle dasselbe meinen:

Bereich	wahr	falsch
Mathematik / Logik	w	f
Englischsprachige Literatur	T (true)	F (false)
Technische Informatik, Schaltalgebra	1	0
Python	True	False
SQL	TRUE	FALSE

In diesem Kurs wird mit 0 und 1 gearbeitet, weil sich Wahrheitstabellen damit am kompaktesten schreiben lassen und weil es der Schreibweise in der technischen Informatik entspricht.

2 – Die drei Grundoperatoren

Aus einfachen Aussagen werden zusammengesetzte Aussagen gebildet. Dafür genügen drei Operatoren: Negation, Konjunktion und Disjunktion. Alle weiteren Verknüpfungen lassen sich aus diesen dreien zusammensetzen.

2.1 Negation (NICHT)

Die Negation dreht den Wahrheitswert um. Schreibweisen: $\neg a$, $\bar{a}$, in Python `not a`, in SQL `NOT a`.

a	$\neg a$
0	1
1	0

Beispiel: a = „Die Rechnung ist bezahlt.“ Dann ist $\neg a$ = „Die Rechnung ist nicht bezahlt.“

2.2 Konjunktion (UND)

Die Konjunktion ist genau dann wahr, wenn **beide** Teilaussagen wahr sind. Schreibweisen: $a \wedge b$, $a \cdot b$, in Python `a and b`, in SQL `a AND b`.

a	b	$a \wedge b$
0	0	0
0	1	0
1	0	0
1	1	1

Merkhilfe: Das UND verhält sich wie die Multiplikation. $0 \cdot 0 = 0$, $0 \cdot 1 = 0$, $1 \cdot 1 = 1$. Deshalb wird es auch als $a \cdot b$ geschrieben. Ein einziger falscher Teil macht das Ganze falsch.

2.3 Disjunktion (ODER)

Die Disjunktion ist wahr, wenn **mindestens eine** Teilaussage wahr ist. Schreibweisen: $a \vee b$, $a + b$, in Python `a or b`, in SQL `a OR b`.

a	b	$a \vee b$
0	0	0
0	1	1
1	0	1
1	1	1

Stolperstelle: Das logische ODER ist ein einschließendes ODER. „Kaffee oder Tee“ bedeutet in der Logik: Kaffee, Tee oder beides. Das umgangssprachliche „entweder ... oder“ ist ein anderer Operator (XOR, Abschnitt 4). Missverständnisse in Fachanforderungen entstehen fast immer an dieser Stelle.

2.4 Rangfolge der Operatoren

Wie in der Arithmetik gibt es eine Bindungsstärke. Sie entspricht genau der Regel „Punkt vor Strich“:

1. $\neg$ (NICHT) bindet am stärksten

2. $\wedge$ (UND) bindet mittel – entspricht dem Malpunkt
3. $\vee$ (ODER) bindet am schwächsten – entspricht dem Pluszeichen

Damit bedeutet $a \vee b \wedge c$ dasselbe wie $a \vee (b \wedge c)$ und nicht $(a \vee b) \wedge c$. Im Zweifel: Klammern setzen. Klammern kosten nichts und verhindern die häufigste Fehlerquelle in SQL-Bedingungen.

3 – Wahrheitstabellen systematisch aufstellen

Eine Wahrheitstabelle listet alle möglichen Belegungen der Variablen und den daraus folgenden Wert des Gesamtausdrucks. Sie ist das zentrale Werkzeug: Damit lässt sich jede Behauptung über boolesche Ausdrücke vollständig überprüfen.

Anzahl der Zeilen

Bei n Variablen gibt es 2^n Kombinationen:

Variablen n	Zeilen 2^n
1	2
2	4
3	8
4	16
5	32

Das ist bereits eine Anwendung der Potenzrechnung von Doc 3 und zugleich der Grund, warum Wahrheitstabellen ab etwa fünf Variablen unpraktisch werden – man rechnet dann mit den Gesetzen aus Abschnitt 5.

Systematik der Belegungen

Damit keine Kombination vergessen wird, werden die Spalten nach einem festen Muster gefüllt: Die rechte Spalte wechselt in jeder Zeile (0,1,0,1,...), die nächste alle zwei Zeilen (0,0,1,1,...), die nächste alle vier Zeilen. Das entspricht dem Hochzählen im Binärsystem.

a	b	c
0	0	0
0	0	1
0	1	0
0	1	1
1	0	0
1	0	1
1	1	0
1	1	1

Auswertung mit Zwischenspalten

Zusammengesetzte Ausdrücke werden schrittweise ausgewertet – für jede Klammer und jede Teilverknüpfung eine eigene Spalte. Beispiel: $(a \wedge b) \vee \neg c$

a	b	c	$a \wedge b$	$\neg c$	$(a \wedge b) \vee \neg c$
0	0	0			
0	0	1			

a	b	c	$a \wedge b$	$\neg c$	$(a \wedge b) \vee \neg c$
0	1	0			
0	1	1			
1	0	0			
1	0	1			
1	1	0			
1	1	1			

Drei Typen von Ausdrücken

- **Tautologie** – in jeder Zeile wahr. Beispiel: $a \vee \neg a$. Eine Bedingung, die immer zutrifft, filtert nichts.
- **Kontradiktion** – in jeder Zeile falsch. Beispiel: $a \wedge \neg a$. Eine Bedingung, die nie zutrifft, liefert null Zeilen.
- **Erfüllbare Aussage** – in mindestens einer Zeile wahr. Der Normalfall.

Praxisbezug: Die klassische leere Ergebnismenge in SQL entsteht durch eine unbeabsichtigte Kontradiktion, etwa `WHERE status = 'offen' AND status = 'bezahlt'`. Eine Spalte kann pro Zeile nur einen Wert haben, die Bedingung ist also nie erfüllbar. Die Wahrheitstabelle zeigt das sofort.

4 – Weitere Verknüpfungen

Die folgenden Operatoren treten in der Praxis häufig auf. Alle lassen sich auf die drei Grundoperatoren zurückführen.

4.1 XOR – das ausschließende ODER

Wahr, wenn **genau eine** der beiden Aussagen wahr ist. Schreibweise: $a \underline{\vee} b$ oder **a XOR b**.

a	b	$a \underline{\vee} b$
0	0	
0	1	
1	0	
1	1	

Zurückführung: $a \underline{\vee} b = (a \vee b) \wedge \neg(a \wedge b)$ – also „mindestens eines, aber nicht beide“. In Python gibt es dafür `!=` auf Wahrheitswerten oder den Operator `^`.

4.2 NAND und NOR

Die Negationen von UND und ODER. In der Schaltungstechnik sind sie zentral, weil sich mit NAND allein jede beliebige Schaltung aufbauen lässt.

a	b	$a \wedge b$	NAND	$a \vee b$	NOR
0	0	0	1	0	1
0	1	0	1	1	0
1	0	0	1	1	0
1	1	1	0	1	0

4.3 Implikation

Die Implikation $a \Rightarrow b$ liest sich als „wenn a, dann b“. Sie ist nur dann falsch, wenn die Voraussetzung wahr ist, die Folgerung aber nicht eintritt.

a	b	$a \Rightarrow b$
0	0	1
0	1	1
1	0	0
1	1	1

Stolperstelle: Bei falscher Voraussetzung ist die Implikation immer wahr. Der Satz „Wenn es heute schneit, gebe ich einen aus“ ist an einem schneefreien Tag nicht gebrochen worden – unabhängig davon, ob ausgegeben wurde. Diese Eigenschaft heißt *ex falso quodlibet* und irritiert regelmäßig. Zurückführung: $a \Rightarrow b = \neg a \vee b$.

4.4 Äquivalenz

Zwei Aussagen sind äquivalent ($a \Leftrightarrow b$), wenn sie denselben Wahrheitswert haben. Das ist die Verneinung von XOR und zugleich das Werkzeug, mit dem in Abschnitt 7 nachgewiesen wird, dass zwei Terme dasselbe bedeuten.

a	b	$a \Leftrightarrow b$
0	0	1
0	1	0
1	0	0
1	1	1

5 – Rechengesetze der Booleschen Algebra

Die Gesetze erlauben es, Ausdrücke umzuformen und zu vereinfachen, ohne ihre Bedeutung zu ändern. Viele sind aus der gewöhnlichen Algebra bekannt – mit zwei wichtigen Abweichungen, die weiter unten markiert sind.

Gesetz	für $\wedge$ (UND)	für $\vee$ (ODER)
Kommutativgesetz	$a \wedge b = b \wedge a$	$a \vee b = b \vee a$
Assoziativgesetz	$(a \wedge b) \wedge c = a \wedge (b \wedge c)$	$(a \vee b) \vee c = a \vee (b \vee c)$
Distributivgesetz	$a \wedge (b \vee c) = (a \wedge b) \vee (a \wedge c)$	$a \vee (b \wedge c) = (a \vee b) \wedge (a \vee c)$
Neutrales Element	$a \wedge 1 = a$	$a \vee 0 = a$
Extremalgesetz	$a \wedge 0 = 0$	$a \vee 1 = 1$
Idempotenzgesetz	$a \wedge a = a$	$a \vee a = a$
Komplementärsgesetz	$a \wedge \neg a = 0$	$a \vee \neg a = 1$
Absorptionsgesetz	$a \wedge (a \vee b) = a$	$a \vee (a \wedge b) = a$
Doppelte Negation	$\neg(\neg a) = a$	$\neg(\neg a) = a$
De Morgan	$\neg(a \wedge b) = \neg a \vee \neg b$	$\neg(a \vee b) = \neg a \wedge \neg b$

Die beiden Abweichungen von der gewohnten Algebra

- Es gibt **zwei** Distributivgesetze. In der gewöhnlichen Algebra gilt $a \cdot (b + c) = ab + ac$, aber **nicht** $a + (b \cdot c) = (a+b) \cdot (a+c)$. In der Booleschen Algebra gelten beide Richtungen.
- Es gibt **kein** Potenzieren und keine Vielfachen. $a \wedge a$ ergibt nicht a^2 , sondern wieder a (Idempotenz). Ebenso $a \vee a = a$. Wiederholung bringt nichts Neues.

Dualitätsprinzip

Die Tabelle ist spaltenweise symmetrisch. Vertauscht man in einem gültigen Gesetz überall $\wedge$ mit $\vee$ und 0 mit 1, entsteht wieder ein gültiges Gesetz. Das halbiert den Lernaufwand: Es genügt, jeweils eine Spalte zu beherrschen.

Praxisbezug: Das Absorptionsgesetz ist der häufigste Vereinfachungsfall in echten SQL-Bedingungen. `WHERE aktiv = 1 OR (aktiv = 1 AND region = 'Nord')` lässt sich vollständig zu `WHERE aktiv = 1` kürzen – der zweite Teil ändert das Ergebnis nicht.

6 – Die Gesetze von De Morgan

Diese beiden Gesetze verdienen einen eigenen Abschnitt, weil sie in der Praxis am häufigsten gebraucht und am häufigsten falsch angewendet werden. Sie beschreiben, wie eine Verneinung in eine Klammer hineingezogen wird.

$$\neg(a \wedge b) = \neg a \vee \neg b$$

$$\neg(a \vee b) = \neg a \wedge \neg b$$

In Worten: Beim Hineinziehen der Verneinung wird jede Teilaussage verneint und der Operator kippt – aus UND wird ODER und umgekehrt.

Nachweis durch Wahrheitstabelle

Die Gleichheit wird bewiesen, indem beide Seiten in dieselbe Tabelle geschrieben werden. Stimmen die Spalten in allen Zeilen überein, sind die Ausdrücke äquivalent.

a	b	$a \wedge b$	$\neg(a \wedge b)$	$\neg a$	$\neg b$	$\neg a \vee \neg b$
0	0					
0	1					
1	0					
1	1					

Die vierte und die siebte Spalte sind identisch. Damit ist das erste Gesetz bewiesen.

Sprachliches Beispiel

Aussage: „Es ist nicht wahr, dass die Datei existiert und lesbar ist.“

Das bedeutet: Die Datei existiert nicht, oder sie ist nicht lesbar. Nicht dagegen: Die Datei existiert nicht und ist nicht lesbar – das wäre eine stärkere Behauptung und ein sehr häufiger Denkfehler.

Praxisbezug: De Morgan ist das Werkzeug, um negierte Filter lesbar zu machen. Statt `NOT (status = 'offen' OR status = 'storniert')` schreibt man `status <> 'offen' AND status <> 'storniert'`. Beide Varianten

liefern dasselbe – die zweite ist für Datenbanken in der Regel besser optimierbar und für Menschen leichter zu prüfen.

7 – Terme vereinfachen

Vereinfachen bedeutet: einen Ausdruck durch einen kürzeren ersetzen, der in jeder Zeile der Wahrheitstabelle denselben Wert liefert. Vorgegangen wird schrittweise, und jeder Schritt wird mit dem verwendeten Gesetz begründet.

Beispiel 1

$$\begin{aligned} & (a \wedge b) \vee (a \wedge \neg b) \\ = & a \wedge (b \vee \neg b) & | \text{ Distributivgesetz, } a \text{ ausgeklammert} \\ = & a \wedge 1 & | \text{ Komplementärgesetz} \\ = & a & | \text{ Neutrales Element} \end{aligned}$$

Interpretation: Ob b zutrifft oder nicht, ist gleichgültig – entscheidend ist allein a.

Beispiel 2

$$\begin{aligned} & \neg(\neg a \vee b) \\ = & \neg(\neg a) \wedge \neg b & | \text{ De Morgan} \\ = & a \wedge \neg b & | \text{ Doppelte Negation} \end{aligned}$$

Beispiel 3

$$\begin{aligned} & (a \vee b) \wedge (a \vee \neg b) \\ = & a \vee (b \wedge \neg b) & | \text{ Distributivgesetz (zweite Form)} \\ = & a \vee \emptyset & | \text{ Komplementärgesetz} \\ = & a & | \text{ Neutrales Element} \end{aligned}$$

Beispiel 4 – Absorption erkennen

$$\begin{aligned} & a \vee (a \wedge b) \vee (a \wedge c) \\ = & a \vee (a \wedge c) & | \text{ Absorptionsgesetz auf die ersten beiden Glieder} \\ = & a & | \text{ Absorptionsgesetz erneut} \end{aligned}$$

Vorgehen: Erst nach Klammern suchen, in denen dieselbe Variable mehrfach vorkommt – dort greifen Distributiv- und Absorptionsgesetz. Dann Negationen mit De Morgan nach innen ziehen. Zum Schluss mit einer Wahrheitstabelle gegenprüfen, dass das Ergebnis wirklich äquivalent ist.

8 – Boolesche Logik in Python

Python bildet die drei Grundoperatoren als Schlüsselwörter ab. Darüber hinaus gibt es einige Eigenheiten, die in der Praxis regelmäßig zu Fehlern führen.

Operatoren

```
a = True
b = False

print(not a)      # False
print(a and b)    # False
print(a or b)     # True
print(a != b)     # True    -> entspricht XOR
print(a ^ b)      # True    -> ebenfalls XOR
```

Wahrheitstabelle automatisch erzeugen

Mit `itertools.product` lässt sich jede Wahrheitstabelle in wenigen Zeilen erzeugen. Das eignet sich gut zur Selbstkontrolle nach dem Rechnen von Hand – nicht als Ersatz dafür.

```
from itertools import product

print('a b c | (a and b) or not c')
for a, b, c in product([0, 1], repeat=3):
    ergebnis = (a and b) or not c
    print(a, b, c, '|', int(ergebnis))
```

Kurzschlussauswertung

Python wertet `and` und `or` von links nach rechts aus und bricht ab, sobald das Ergebnis feststeht. Bei `a and b` wird `b` gar nicht erst ausgewertet, wenn `a` bereits falsch ist.

```
# Ohne Kurzschluss würde die Division durch null einen Fehler werfen.
anzahl = 0
if anzahl != 0 and summe / anzahl > 100:
    print('Durchschnitt über 100')

# Die Reihenfolge ist hier nicht vertauschbar:
# summe / anzahl > 100 and anzahl != 0 -> ZeroDivisionError
```

Merken: Das Kommutativgesetz gilt in der Booleschen Algebra immer, in Python aber nur für das Ergebnis – nicht für das Laufzeitverhalten. Reihenfolge kann über Fehler oder kein Fehler entscheiden.

Wahrheitswert beliebiger Objekte

In Bedingungen wandelt Python jeden Wert in einen Wahrheitswert um. Als falsch gelten:

```
False, None, 0, 0.0, '', [], {}, set()

# Alles andere gilt als wahr:
bool('0')      # True -> nicht leerer String!
bool([0])     # True -> Liste mit einem Element
bool(-1)      # True -> jede Zahl ungleich null
```

Dass der String `'0'` als wahr gilt, ist eine klassische Fehlerquelle beim Einlesen von CSV-Dateien, in denen Wahrheitswerte als Text stehen.

Verkettete Vergleiche

```
# Python erlaubt die mathematische Schreibweise:
if 0 <= wert <= 100:
    print('gültiger Prozentwert')
```

```
# Gleichbedeutend, aber länger:
if wert >= 0 and wert <= 100:
    print('gültiger Prozentwert')
```

9 – Boolesche Logik in SQL

In SQL steckt die Boolesche Algebra in der WHERE-Klausel, in HAVING und in CHECK-Bedingungen. Der wichtigste Unterschied zur reinen Logik: SQL kennt drei Wahrheitswerte.

Operatorrangfolge in WHERE

```
-- AND bindet stärker als OR. Diese beiden Abfragen sind verschieden:

SELECT * FROM bestellung
WHERE status = 'offen' OR status = 'neu' AND betrag > 100;
-- gelesen als: status='offen' OR (status='neu' AND betrag>100)

SELECT * FROM bestellung
WHERE (status = 'offen' OR status = 'neu') AND betrag > 100;
-- vermutlich das Gemeinde
```

Regel: Sobald AND und OR in derselben Bedingung vorkommen, werden Klammern gesetzt – auch dann, wenn sie rechnerisch nicht nötig wären. Das ist keine Stilfrage, sondern Fehlervermeidung.

Dreiwertige Logik: der Wert NULL

NULL bedeutet in SQL nicht null und nicht leer, sondern unbekannt. Ein Vergleich mit einem unbekannten Wert liefert weder wahr noch falsch, sondern UNKNOWN. Damit verlässt SQL die Zweiwertigkeit.

a	b	a AND b	a OR b
TRUE	NULL	UNKNOWN	TRUE
FALSE	NULL	FALSE	UNKNOWN
NULL	NULL	UNKNOWN	UNKNOWN

Die WHERE-Klausel liefert nur Zeilen zurück, deren Bedingung TRUE ergibt. UNKNOWN wird wie FALSE behandelt – die Zeile fällt heraus.

```
-- Liefert KEINE Zeilen, auch wenn rabatt in vielen Zeilen NULL ist:
SELECT * FROM artikel WHERE rabatt = NULL;

-- Richtig:
SELECT * FROM artikel WHERE rabatt IS NULL;

-- Ebenfalls unvollständig: Zeilen mit rabatt IS NULL fehlen im Ergebnis.
SELECT * FROM artikel WHERE rabatt <> 10;

-- Vollständig:
SELECT * FROM artikel WHERE rabatt <> 10 OR rabatt IS NULL;
```

Konsequenz: Das Komplementärgesetz $a \vee \neg a = 1$ gilt in SQL nicht mehr. Ist a gleich UNKNOWN, ist auch a OR NOT a gleich UNKNOWN. Wer mit NULL-Werten arbeitet, muss sie in der Bedingung immer ausdrücklich behandeln.

10 – Boolesche Logik in Pandas

Beim Filtern von DataFrames wird nicht ein einzelner Wahrheitswert ausgewertet, sondern eine ganze Spalte gleichzeitig. Dafür gelten andere Operatoren.

Bedeutung	Python (einzeln)	Pandas (Spalte)
UND	and	&
ODER	or	
NICHT	not	~

```
import pandas as pd

# Klammern sind zwingend: & bindet stärker als der Vergleich >
auswahl = df[(df['umsatz'] > 1000) & (df['region'] == 'Nord')]

# Negation einer Bedingung
auswahl = df[~(df['status'] == 'storniert')]

# De Morgan angewendet - gleichbedeutend und meist lesbarer:
auswahl = df[(df['status'] != 'offen') & (df['status'] != 'neu')]
# statt: df[~((df['status'] == 'offen') | (df['status'] == 'neu'))]
```

Die Verwendung von and, or oder not auf einer Spalte führt zu einem ValueError mit dem Hinweis auf die Mehrdeutigkeit des Wahrheitswerts. Der Grund: Eine Spalte mit vielen Werten hat keinen einzelnen Wahrheitswert.

11 – Übungen

Übung 1 – Aussagen erkennen

Zehn Sätze werden daraufhin geprüft, ob sie Aussagen im Sinne der Logik sind. Für die Nicht-Aussagen wird jeweils eine überprüfbare Fassung formuliert. Bezug: Abschnitt 1.

Übung 2 – Wahrheitstabellen zu zwei Variablen

<https://academo.org/demos/logic-gate-simulator/>

Simulator Block

Bezug: Abschnitte 2 und 3.

Übung 3 – Wahrheitstabellen zu drei Variablen

<https://inf-schule.de/rechner/digitaltechnik/Schaltnetze/Uebungen>

Aufg. „Jury-Abstimmung“

Bezug: Abschnitt 3.

Übung 4 – De Morgan anwenden

<https://inf-schule.de/rechner/digitaltechnik/Schaltnetze vereinfachen/Uebungen>

Aufg. (De Morgan) + Aufg. 3 (Distributiv)

Bezug: Abschnitt 6.

Übung 5 – Terme vereinfachen

<https://inf-schule.de/rechner/digitaltechnik/Schaltnetze vereinfachen/Uebungen>

Aufgabe 1 „Kaffeemaschine“

Bezug: Abschnitte 5 und 7.

Übung 6 – Fachanforderung in Logik übersetzen

<https://inf-schule.de/rechner/digitaltechnik/Schaltnetze/Uebungen>

Aufgabe „Tresorraum“ + „Kaffeemaschine“

Bezug: Abschnitte 2.3 und 9.

Übung 7 – Fehlersuche

Sechs fehlerhafte Bedingungen aus Python, SQL und Pandas werden analysiert und korrigiert. Enthalten sind: fehlende Klammern bei AND/OR, Vergleich mit NULL, and statt & in Pandas, Kurzschluss in falscher Reihenfolge. Bezug: Abschnitte 8, 9 und 10.

Ergänzend aus dem Cornelsen Trainingskurs Mathematik

Die folgenden Aufgaben behandeln nicht die Boolesche Algebra, trainieren aber genau die Umformungsmuster, die in Abschnitt 7 gebraucht werden – Ausklammern, Ausmultiplizieren und das Erkennen gemeinsamer Faktoren. Sie eignen sich als Einstiegsübung für Teilnehmende, denen das Umformen von Termen schwerfällt.

Seite	Aufgaben	Thema	Bezug
S. 6	Aufg. 3, 4, 5	Testen Sie sich: Klammern	Einstiegsdiagnose
S. 9	Aufg. 2, 3	Klammern auflösen und vereinfachen	Distributivgesetz
S. 10	Aufg. 16, 17, 18	Faktoren ausklammern	Distributivgesetz, rückwärts

Seite	Aufgaben	Thema	Bezug
S. 10	Aufg. 14	Identische Terme erkennen	Äquivalenz zweier Terme

Lösungen zu den Buchaufgaben: S. 69 ff.

12 – Anschluss an Doc 2

Die Mengenlehre am folgenden Doc ist dieselbe Struktur in anderer Schreibweise. Diese Entsprechung wird zu Beginn Notebook 2 ausdrücklich hergestellt:

Boolesche Algebra	Mengenlehre	SQL
$a \wedge b$	$A \cap B$ (Schnittmenge)	INNER JOIN, AND
$a \vee b$	$A \cup B$ (Vereinigung)	UNION, OR
$\neg a$	$\bar{A}$ (Komplement)	NOT, EXCEPT
De Morgan	$\neg(A \cap B) = \bar{A} \cup \bar{B}$	negierte Filter
0 (immer falsch)	$\emptyset$ (leere Menge)	leere Ergebnismenge